Sistema de Gestión Documental

Realizado por:
Dayana Sofía Orozco Mendóza

Modelo Documental

ÍNDICE

03 Resumen del modelo

05 Colecciones y documentos

09 Documentos embebidos vs. referencias

11 Patrones de diseño aplicados

13 Reglas de consistencia y cómo se garantizan

14 Índices

16 Consultas y agregaciones de los reportes

18 Carga inicial

Colección	Qué guarda	Crece con
users	Cuentas, rol, estado y espacio usado	Usuarios (pocos)
folders	Jerarquía de carpetas	Carpetas
files	Metadatos de cada archivo (nombre, tamaño, ubicación, dueño...)	Archivos
shares	Una compartición = un recurso + un usuario + un permiso	Comparticiones
comments	Comentarios sobre archivos	Comentarios
activity_log	Historial de operaciones (auditoría)	Cada operación
contenido.files / **contenido.chunks**	Contenido de los archivos (GridFS)	Archivos

2. COLECCIONES Y DOCUMENTOS

2.1 users

json


```
{
 "_id": "ObjectId",
 "nombre": "Ana López",
 "email": "ana@docs.com",
 "passwordHash": "$2a$10$...",
 "rol": "user",
 "activo": true,
 "almacenamientoUsado": 20787,
 "almacenamientoAsignado": 104857600,
 "createdAt": "2026-08-20T15:10:00Z"
 }
```


- email se guarda en minúsculas y es único (índice único).
- passwordHash es un hash bcrypt; nunca se guarda la contraseña.
- rol es admin o user.
- activo permite desactivar sin borrar: el usuario no puede iniciar sesión, pero sus archivos, carpetas e historial se conservan.
- almacenamientoUsado (bytes) es un contador que se actualiza al subir, editar y eliminar definitivamente.
- almacenamientoAsignado (bytes) es la cuota de espacio del usuario. Disponible = asignado − utilizado; el servidor rechaza subidas o ediciones que no quepan. Lo asigna el administrador; los usuarios sin el campo reciben la cuota por defecto (DEFAULT_QUOTA_MB).

3. DOCUMENTOS EMBEBIDOS VS. REFERENCIAS

3.1 Qué está embebido y por qué

Dato embebido	Dónde	Justificación
details (objeto libre)	activity_log	Cada acción tiene datos distintos (tamaño, destinatario, permiso anterior...). Un objeto embebido evita una colección por tipo de acción y se lee siempre junto con el evento. Es la flexibilidad que ofrece un modelo documental.
ancestors (arreglo de ObjectId)	folders	Se consulta siempre junto con la carpeta y su tamaño está acotado por la profundidad del árbol.
metadata.ownerId	contenido.files	Metadato propio de GridFS.
Copias userName y resourceName	*activity_log, comments*	Desnormalización intencional (ver 3.3).

3.2 Qué se representa por referencia y por qué

Relación	Por qué referencia y no embebido	Relación
files.ownerId, folders.ownerId → users	El usuario existe de forma independiente; embeber su datos duplicaría información modificable.	files.ownerId, folders.ownerId → users
files.folderId, folders.parentId → folders	Un archivo cambia de carpeta con frecuencia (mover). Con referencia es una sola actualización; si los archivos estuvieran embebidos en la carpeta habría que copiarlos de un documento a otro.	files.folderId, folders.parentId → folders
files.gridFsId → GridFS	Separar metadatos de contenido permite listar y filtrar sin leer los bytes.	files.gridFsId → GridFS
shares como colección propia	*Ver 3.3, es la decisión más importante.*	shares como colección propia
comments como colección propia	Crecen sin límite por archivo; embebidos podrían acercarse al límite de 16 MB por documento y obligarían a reescribir el archivo en cada comentario	comments como colección propia
activity_log como colección propia	Es append-only, crece sin límite y debe sobrevivir a la eliminación de los recursos (auditoría).	activity_log como colección propia
Relación	*Por qué referencia y no embebido*	Relación

4. PATRONES DE DISEÑO APLICADOS

4.1 Jerarquía de carpetas: referencia al padre + arreglo de ancestros

Cada carpeta guarda parentId (padre inmediato) y ancestors (camino completo).

- Descendientes de una carpeta: una consulta, folders.find({ ancestors: id }), aprovechando el índice multikey sobre ancestors.
- Detección de ciclos: al mover la carpeta A dentro de B, se rechaza si B == A o si A está en B.ancestors. Es una comprobación en memoria, sin recorrer el árbol.
- Costo: al mover una carpeta hay que reescribir ancestors de todos sus descendientes (reparentFolder en utils/tree.js, con un bulkWrite). Se aceptó porque mover carpetas es raro frente a listar y navegar.

4.2 Permisos con herencia

El nivel de acceso de un usuario sobre un recurso se calcula en utils/access.js (getLevel):

1. Si es el dueño, nivel owner.
2. Si no, se buscan en shares los registros del usuario sobre el recurso o sobre cualquiera de sus carpetas ancestro.
3. Se toma el nivel más alto: read < comment < edit < owner.

La herencia se calcula al leer, por lo que no hay que propagar permisos a cada archivo al compartir una carpeta.

5. REGLAS DE CONSISTENCIA Y CÓMO SE GARANTIZAN

Regla del enunciado	Mecanismo
Sin ciclos ni referencias inválidas	Validación con ancestors al mover (routes/folders.js); ancestors se recalcula con reparentFolder.
Solo acceso autorizado	loadResource + getLevel en cada ruta; los botones del cliente solo ocultan opciones, la autoridad es el servidor.
Nombres sin duplicar en una carpeta	nameTaken antes de crear, renombrar y mover.
Un recurso no se comparte dos veces con la misma persona	Índice único en shares.
Papelera fuera de las consultas normales	Filtro deletedAt: null en todas las consultas.
Restaurar mantiene la jerarquía	Si la carpeta original ya no existe o está en la papelera, el recurso vuelve a la raíz (reparentFolder); si el nombre choca, se renombra (restaurado).
Eliminar definitivo mantiene la consistencia	Borra contenido en GridFS, metadatos, shares y comments asociados; descuenta almacenamientoUsado; conserva activity_log.
Desactivar usuario sin perder historial	Solo cambia activo; el middleware rechaza el token de usuarios inactivos.
Historial conservado	activity_log nunca se borra; guarda copia del nombre del recurso.

8. CARGA INICIAL

npm run seed (en servidor/) elimina la base de datos configurada y la recrea con:

Elemento	Cantidad
Usuarios	3 (1 administrador y 2 usuarios)
Carpetas	22 (7 por usuario más "Borradores")
Archivos	50 (.txt y .md), contenido guardado en GridFS
Comparticiones	32 (28 de archivos y 4 de carpetas)
Comentarios	5
En papelera	6 archivos y 1 carpeta
Historial	Más de 200 eventos repartidos en unos 40 días (el número exacto lo imprime npm run seed)

De los 50 archivos, 6 están en la papelera, por lo que en pantalla se ven 44. La generación usa una semilla fija, así que los datos son reproducibles.